

Contenido

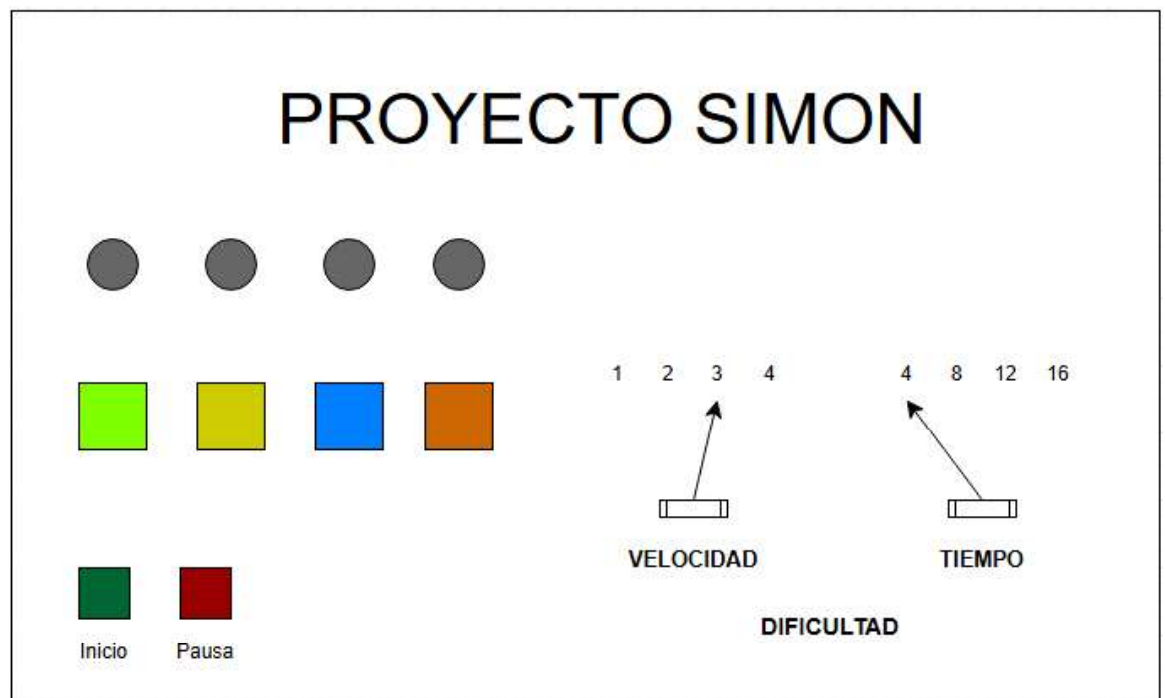
Descripción general.....	4
Boceto.....	4
Diseño físico.....	4
Lista de materiales.....	4
Diseño del software	5
Requisitos funcionales.....	5
Estados del Juego.....	6
Estado 0 — “Esperar Inicio”	6
Estado 1 — Secuencia de inicio.....	6
Estado 2 — “Mostrar Secuencia al Jugador”	6
Estado 3 — “Lectura de Pulsaciones”	6
Estado 4 — “Pausa Breve”	6
Estado 5 — “Pausa Manual”	6
Estado 6 — “Fase Superada”	6
Estado 7 — “Victoria”	7
Matriz de cambio de estados.....	7
Diagrama de estados.....	7
Tabla de variables.....	7
Objetos (instancias de clases)	7
Variables globales y constantes	8
Hilos de Ejecución.....	8
Tabla de hilos	8
Tabla actividades del Simon	8
hiloEstado0	9
hiloDial1 (Dial de Tiempo).....	9
hiloDial2 (Dial de Velocidad).....	9
hiloEstado1	10
hiloEstado2	10
hiloEstado3	11
hiloEstado4	12
hiloEstado5	12
hiloEstado6	13
hiloEstado7	13

Lista de funciones de manejadores de estado	14
Lista de manejadores de eventos GPIO	14
Descripción de objetos y clases propias del proyecto.....	15
Implementación	17
Tabla conexiones BBB.....	17
Circuitos electrónicos.....	17
Leds + botones.....	18
Zumbador.....	18
Diales.....	18
Preparación de la parte electronica y la caja	19
Preparación de la electrónica durante la fase de desarrollo	19
Preparación de los componentes y la caja.....	22
Preparación final de la caja	24
Pruebas de software	25
Problemas encontrados.....	26
Resultado final.....	27
Tabla seguimiento del desarrollo.....	30

Descripción general

El proyecto es el juego clásico **Simon** haciendo uso de una BeagleBone Black. Se usan seis botones arcade con LED para los colores y funciones START/PAUSA, un zumbador para los tonos, y dos diales para ajustar velocidad y tiempo de respuesta. La lógica se organiza en nueve estados (inicio, secuencia de inicio, reproducción de la secuencia de juego, lectura de los botones, pausas, cambio de fase y victoria), cada uno en su propio hilo, mientras interrupciones GPIO manejan los botones de inicio y pausa. Así, el juego avanza y valida la secuencia de colores de forma concurrente y configurable en tiempo real.

BOCETO



Diseño físico

LISTA DE MATERIALES

Componente	Especificaciones	Cantidad
BeagleBone Black (BBB)	—	1
Fuente de alimentación BBB	5 V @ 2 A	1
Fuente de alimentación externa	5 V @ 1 A	1
Protoboard	—	2
Botón arcade (con LED interno)	—	6
Potenciómetro (dial)	—	2
Servomotor	—	2
Zumbador (buzzer)	—	1
Transistor NPN	—	1
MOSFET	—	6
Resistencia	5 k Ω	7
Resistencia	220 Ω	7

Resistencia	1 k Ω	6
Resistencia	10 k Ω	2
Condensador	100 nF	6

Diseño del software

REQUISITOS FUNCIONALES

1. Ejecución automática
 - El programa debe arrancar automáticamente al encender la BeagleBone Black.
2. Arranque de partida
 - Al pulsar el botón START, el sistema inicia una nueva partida: reinicia la secuencia, reproduce confirmación y cambia al estado “Secuencia de inicio de juego” (Estado 1).
3. Secuencia de inicio de juego
 - En Estado 1 muestra todos los LEDs, luego cada color con su sonido, y da un feedback visual/sonoro antes de iniciar la partida real.
4. Visualización de secuencia al jugador
 - En Estado 2 reproduce la secuencia actual de colores y tonos a la velocidad definida por el dial de velocidad.
5. Lectura y validación de pulsaciones

En Estado 3, el sistema espera la pulsación de un botón.

 - Si el color pulsado coincide con el siguiente color de la secuencia, continúa con la siguiente comprobación.
 - Si el color no coincide o se supera el tiempo máximo de espera (timeout), señala fallo y reinicia el juego.
6. Gestión de fallos y éxitos
 - En caso de fallo: reproduce sonido de error, reinicia el juego y vuelve al Estado 0.
 - En caso de éxito: añade un nuevo color, reproduce sonido de acierto y, según el nivel:
 - a) Si nivel es múltiplo de 5 es un cambio de fase, y por tanto cambia al Estado 6.
 - b) Si se ha superado el nivel máximo que se ha programado para que sea el 20, cambia al Estado 7.
 - c) En otro caso, cambia al Estado 4 de pausa.
7. Pausa breve y manual
 - Estado 4: parpadeo de LED PAUSE durante 1,5 s, luego vuelve al Estado 2.
 - Estado 5: pausa indefinida activada por botón PAUSE, con parpadeo de LED PAUSE; reanuda al pulsar PAUSE otra vez (Estado 2).
8. Cambio de estado 6 y 7

- Estado 6: reproduce la melodía de fase extra y parpadea todos los LEDs tres veces; luego vuelve al Estado 4.
 - Estado 7: reproduce la melodía de victoria y parpadea todos los LEDs cuatro veces; reinicia la secuencia y vuelve al Estado 0.
9. Ajuste dinámico de parámetros
- En Estados 0 y 5 lee continuamente los dos diales para ajustar nivel de tiempo y nivel de velocidad.

ESTADOS DEL JUEGO

Estado 0 — “Esperar Inicio”

- Descripción: Modo de reposo; parpadeo del LED de inicio y reproducción continua de distintas melodías que se reproducen aleatoriamente hasta que se pulse el botón START.
- Transición: Al pulsar START en el callback de hardware, reinicia secuencia y pasa a Estado 1.

Estado 1 — Secuencia de inicio

- Descripción: Muestra una secuencia de inicio con todos los LEDs encendidos, luego uno a uno con sonido, y un feedback final antes de empezar el juego.
- Transición: Al finalizar la secuencia de inicio, cambia automáticamente a Estado 2.

Estado 2 — “Mostrar Secuencia al Jugador”

- Descripción: Reproduce la secuencia de colores guardada (LED + tono) a la velocidad ajustada por el dial.
- Transición: Al terminar, pasa a Estado 3 (lectura de jugador).

Estado 3 — “Lectura de Pulsaciones”

- Descripción: Espera y valida las pulsaciones del jugador respecto a la secuencia haciendo uso de polling. Controla timeout según dial de tiempo.
 - En caso de fallo: toca sonido de error, reinicia secuencia y va al Estado 0.
 - En caso de acierto: añade un color y:
 - Si llega a múltiplos de 5 (y < máximo): Estado 6 (fase extra).
 - Si llega al nivel máximo: Estado 7 (victoria).
 - En otro caso: Estado 4 (pausa breve).

Estado 4 — “Pausa Breve”

- Descripción: Parpadeo del LED de pausa durante un tiempo fijo (≈ 1.5 s), luego regresa a Estado 2 para mostrar la siguiente ronda.

Estado 5 — “Pausa Manual”

- Descripción: Pausa indefinida activada por el botón PAUSA; muestra LEDs de juego y parpadea LED PAUSA hasta reanudar.
- Transición: El callback de PAUSA cambia entre 5 y 2.

Estado 6 — “Fase Superada”

- Descripción: Toca una pequeña melodía y parpadea todos los LEDs 3 veces como celebración de fase superada.
- Transición: Luego vuelve automáticamente a Estado 4 para la siguiente secuencia.

Estado 7 — “Victoria”

- Descripción: Melodía de victoria y parpadeo de todos los LEDs 4 veces; reinicia secuencia y regresa a Estado 0.

MATRIZ DE CAMBIO DE ESTADOS

Estado origen	Evento disparador	Acción	Estado destino
0	Pulsación START (GPIO)	secuencia.reiniciar(), secuencia.añadirColor(), parar melodía intro	1
1	Fin de secuencia de inicio	—	2
2	Fin de mostrar secuencia de juego	—	3
3 (fallo)	Botón incorrecto o timeout	buzzer.reproducirFallo(), secuencia.reiniciar()	0
3 (acierto)	Pulsación correcta	buzzer.reproducirAcierto(), secuencia.añadirColor()	4 / 6 / 7
4	Timeout 1,5 s	—	2
4	Pulsación PAUSE (GPIO)	—	5
5	Pulsación PAUSE (GPIO)	leds.apagarTodos()	2
6	Fin de celebración fase extra	—	4
7	Fin de celebración victoria	secuencia.reiniciar()	0

DIAGRAMA DE ESTADOS

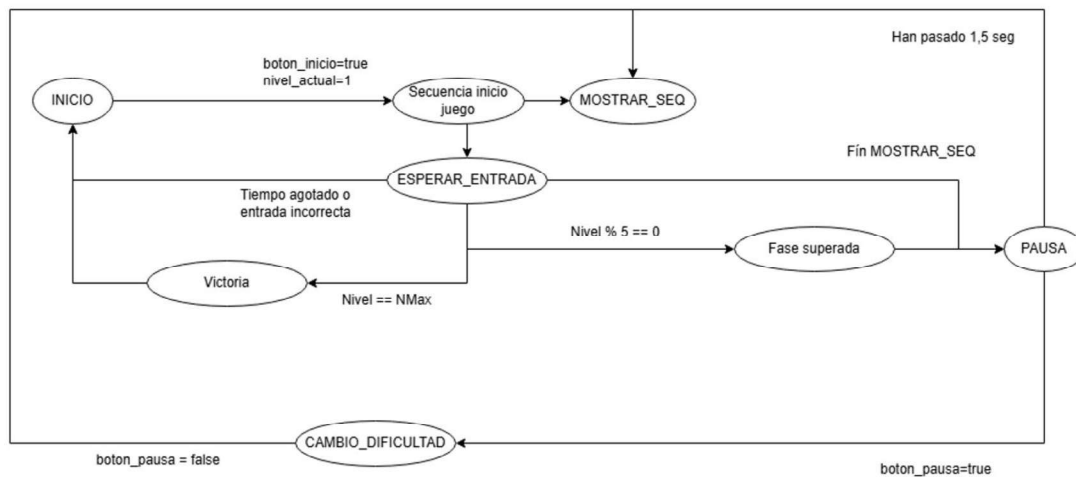


TABLA DE VARIABLES

Objetos (instancias de clases)

Nombre	Tipo	Descripción
stateMgr	StateMonitor	Gestor central de estados; sincroniza hilos y controla fases.
botones	SimonBotones	Interfaz de lectura de botones del SIMON, START y PAUSE.
leds	SimonLeds	Interfaz de control de LEDs de colores y LEDs START/PAUSE.
buzzer	SimonBuzzer	Generador de sonidos y melodías mediante PWM.
secuencia	SimonSecuencia	Almacena y gestiona la secuencia de colores del juego.
dialVelocidad	SimonDial	Potenciómetro para ajustar la velocidad de reproducción.
dialTiempo	SimonDial	Potenciómetro para ajustar el timeout de pulsaciones.

Variables globales y constantes

Nombre	Tipo	Descripción
nivelVelocidad	int	Índice (0–3) leído de dialVelocidad para velDelay[].
nivelTiempo	int	Índice (0–3) leído de dialTiempo para tmoBtn[].
parpadeoHigh	int (µs)	Duración de encendido en parpadeos de LEDs (estados 0, 4, 5).
parpadeoLow	int (µs)	Duración de apagado en parpadeos de LEDs (estados 0, 4, 5).
velDelay[4]	unsigned (ms)	Retrasos de sonido y LED según nivel de velocidad: {300,250,150,80}.
tmoBtn[4]	unsigned (ms)	Timeouts de pulsación según nivel de tiempo: {5000,4000,3000,2000}.
SEQ_MAX	int	Nivel máximo de la secuencia (secuencia.getNivelMax()).

HILOS DE EJECUCIÓN

Tabla de hilos

Hilo	Función	Estado(s) activos	Hilos internos
hiloEstado0	Lógica de Esperar Inicio	0	Crea tMelodia para reproducir la melodía de espera dentro del propio hilo
hiloDial1	Lectura y ajuste de dialTiempo	0, 5	—
hiloDial2	Lectura y ajuste de dialVelocidad	0, 5	—
hiloEstado1	Demo inicial de secuencia	1	—
hiloEstado2	Mostrar secuencia al jugador	2	—
hiloEstado3	Lectura y validación de pulsaciones del jugador	3	—
hiloEstado4	Parpadeo de pausa breve	4	—
hiloEstado5	Pausa manual (LEDs y espera de reanudar)	5	—
hiloEstado6	Celebración fase superada	6	—
hiloEstado7	Celebración victoria	7	—

Tabla actividades del Simon

Actividad	Estados en los que se ejecuta	Hilo responsable
Esperar inicio del juego	0	hiloEstado0
Reproducir melodía de espera	0	Interno en hiloEstado0
Leer dial de tiempo	0, 5	hiloDial1
Leer dial de velocidad	0, 5	hiloDial2
Reproducir secuencia de inicio	1	hiloEstado1
Mostrar secuencia de colores	2	hiloEstado2
Leer pulsaciones del jugador	3	hiloEstado3
Validar secuencia y nivel	3	hiloEstado3

Parpadear LED de pausa breve	4	hiloEstado4
Mantener pausa manual	5	hiloEstado5
Celebrar fase superada	6	hiloEstado6
Celebrar victoria	7	hiloEstado7
Reiniciar partida	7	hiloEstado7

hiloEstado0

1. **Espera activa:**
 - Llama a stateMgr.waitState(cfg) y bloquea hasta que el estado sea 0.
2. **Melodía de espera (hilo anidado):**
 - Crea tMelodia, que en bucle llama a buzzer.reproducirMelodiasIntro() mientras el estado siga siendo 0.
3. **Parpadeo LED START:**
 - Alterna encendido/apagado del LED START con duraciones parpadeoHigh y parpadeoLow.
4. **Finalización:**
 - Cuando cambia el estado (por pulsación), hace pthread_join(tMelodia) para terminar la melodía y apaga el LED START.
5. **Pseudocódigo**

Esperar a que el estado actual sea 0
Mientras el estado sea 0:
 Parpadear LED START (encendido/apagado)
 Reproducir melodía de fondo (hilo interno)
Al cambiar el estado, detener la melodía y apagar el LED

hiloDial1 (Dial de Tiempo)

- **En estados 0 y 5:**
 - Cada 25 ms llama a dialTiempo.actualizar() para leer el potenciómetro y ajustar nivelTiempo.
- **Pseudocódigo**

Si el estado actual es 0 o 5:
Cada 25 ms leer el valor del potenciómetro
Actualizar el nivelTiempo (0 a 3)

hiloDial2 (Dial de Velocidad)

- **En estados 0 y 5:**

- Cada 25 ms llama a `dialVelocidad.actualizar()` para leer el potenciómetro y ajustar `nivelVelocidad`.

- **Pseudocódigo**

Si el estado actual es 0 o 5:
 Cada 25 ms leer el valor del potenciómetro
 Actualizar el `nivelVelocidad` (0 a 3)

hiloEstado1

1. **Espera:** bloquea hasta que el estado sea 1.
2. **Secuencia de LEDs para indicar el inicio del juego:**
 - Enciende todos los LEDs (`leds.encenderLedsJuego()`), espera 350 ms, luego apaga todos y espera 250 ms.
 - Recorre los cuatro colores: enciende cada LED, toca su tono (duración fija), pequeña pausa, apaga, otra pausa.
3. **Feedback final:**
 - Enciende todos los LEDs y llama a `buzzer.reproducirInicio()`, espera 300 ms, apaga todo.
4. **Transición:**
 - Llama a `stateMgr.changeState(2)`.
5. **Pseudocódigo**

Esperar a que el estado actual sea 1
 Encender todos los LEDs de juego durante 350 ms
 Apagar todos los LEDs durante 250 ms
 Para cada color (rojo, verde, amarillo, azul):
 Encender LED
 Reproducir tono asociado
 Apagar LED tras breve pausa
 Encender todos los LEDs
 Reproducir sonido de inicio
 Apagar todos los LEDs
 Cambiar a estado 2

hiloEstado2

1. **Espera:** hasta que el estado sea 2.
2. **Lectura de velocidad:** actualiza `nivelVelocidad = dialVelocidad.getNivel()`.
3. **Mostrar secuencia:**
 - Itera sobre `secuencia.getNivelActual()`:

- Para cada color, enciende su LED, reproduce tono con duración `velDelay[nivelVelocidad]`, espera `velDelay` ms, apaga LED, espera `velDelay/2` ms.
- 4. **Transición:**
 - Llama a `stateMgr.changeState(3)`.

5. Pseudocódigo

Esperar a que el estado actual sea 2
 Leer nivel de velocidad desde el dial
 Para cada color en la secuencia actual:
 Encender LED correspondiente
 Reproducir tono con duración según `velDelay[nivel]`
 Esperar
 Apagar LED
 Pausa breve entre colores
 Cambiar a estado 3

hiloEstado3

1. **Espera:** hasta que el estado sea 3.
2. **Lectura de tiempo:** actualiza `nivelTiempo = dialTiempo.getNivel()`.
3. **Validación de pulsaciones:**
 - Para cada color en la secuencia:
 - Mide tiempo con `steady_clock` y busca lectura de botón dentro de `tmoBtn[nivelTiempo]` ms.
 - En caso de pulsación: reproduce tono corto; si el botón no coincide con el color esperado, marca fallo.
 - Si expira el timeout o hay fallo, sale de bucle.
4. **Resultado:**
 - **Fallo:** `buzzer.reproducirFallo()`, `secuencia.reiniciar()`, va a Estado 0.
 - **Acierto:** `buzzer.reproducirAcierto()`, `secuencia.añadirColor()`, y según el nivel actual:
 - Si `nivel % 5 == 0` y $< \text{máximo}$ → Estado 6.
 - Si $\text{nivel} \geq \text{máximo}$ → Estado 7.
 - En otro caso → Estado 4.
5. **Pseudocódigo**

Esperar a que el estado actual sea 3
 Leer nivel de tiempo desde el dial

```

Para cada color en la secuencia actual:
  Iniciar temporizador
  Esperar pulsación válida (hasta timeout)
  Si botón pulsado es correcto:
    Reproducir tono del botón
    Esperar a que se suelte
  Si botón incorrecto o timeout:
    Reproducir tono de fallo
    Reiniciar secuencia
    Cambiar a estado 0
    Salir

Si la secuencia fue pulsada correctamente:
  Añadir nuevo color
  Reproducir sonido de acierto
  Si se alcanzó un múltiplo de 5 y aún no es el nivel máximo:
    Cambiar a estado 6 (fase superada)
  Si se alcanzó el nivel máximo:
    Cambiar a estado 7 (victoria)
  Si no:
    Cambiar a estado 4 (pausa breve)

```

hiloEstado4

1. **Espera:** hasta que el estado sea 4.
2. **Parpadeo de pausa:**
 - Muestra LED PAUSE con parpadeoHigh/Low en bucle.
 - Controla un timeout de 1.5 s; al expirar, cambia a Estado 2.
3. **Pseudocódigo**

```

Esperar a que el estado actual sea 4
Durante 1.5 s:
  Parpadear LED PAUSE
Al finalizar el tiempo, cambiar a estado 2

```

hiloEstado5

1. **Espera:** hasta que el estado sea 5.
2. **Mantener LEDs de juego:** enciende todos los LEDs de colores fijos.
3. **Parpadeo PAUSE:** parpadea LED PAUSE igual que en el estado 4.
4. **Salida:** cuando el callback de PAUSA cambia el estado, el hilo vuelve a bloquearse y los LEDs se apagan en el callback.
5. **Pseudocódigo**

Esperar a que el estado actual sea 5 Encender LEDs de juego Mientras el estado siga siendo 5: Parpadear LED PAUSE Cuando cambie el estado, apagar todos los LEDs
--

hiloEstado6

1. **Espera:** hasta que el estado sea 6.
2. **Celebración de fase:**
 - `buzzer.reproducirFaseExtra()`.
 - Tres ciclos de encendido/apagado de todos los LEDs con pausas de 150 ms.
3. **Transición:** vuelve a Estado 4.
4. **Pseudocódigo**

Esperar a que el estado actual sea 6 Reproducir sonido de fase superada Durante 3 ciclos: Encender todos los LEDs Esperar 150 ms Apagar todos los LEDs Esperar 150 ms Cambiar a estado 4

hiloEstado7

1. **Espera:** hasta que el estado sea 7.
2. **Victoria:**
 - `buzzer.reproducirVictoria()`.
 - Cuatro ciclos de todos los LEDs parpadeando con pausas de 300 ms.
3. **Reinicio:**
 - `secuencia.reiniciar()`.
 - Pasa a Estado 0 para nuevo juego.
4. **Pseudocódigo**

Esperar a que el estado actual sea 7 Reproducir sonido de victoria Durante 4 ciclos: Encender todos los LEDs Esperar 300 ms Apagar todos los LEDs
--

Esperar 300 ms Reiniciar secuencia Cambiar a estado 0

LISTA DE FUNCIONES DE MANEJADORES DE ESTADO

Estas funciones se invocan dentro de cada hilo al detectar su estado asignado:

- hiloEstado0() → maneja Estado 0 (“Esperar Inicio”).
- hiloEstado1() → Estado 1 (“Secuencia de inicio de la partida”).
- hiloEstado2() → Estado 2 (“Mostrar Secuencia”).
- hiloEstado3() → Estado 3 (“Lectura de Pulsaciones”).
- hiloEstado4() → Estado 4 (“Pausa Breve”).
- hiloEstado5() → Estado 5 (“Pausa Manual”).
- hiloEstado6() → Estado 6 (“Fase Superada”).
- hiloEstado7() → Estado 7 (“Victoria”).

Para cada estado el pseudocódigo podría ser:

```
// LISTA DE FUNCIONES DE MANEJADORES DE ESTADO
// Cada función hiloEstadoN() implementa el siguiente patrón:
wait hasta que stateMgr.getState() == N
  ejecutar lógica específica de Estado N    ← ver pseudocódigo detallado en la sección
  correspondiente
  si corresponde, llamar a stateMgr.changeState(M)
repetir

hiloEstado0() → Estado 0 “Esperar Inicio”
hiloEstado1() → Estado 1 “Secuencia de inicio de la partida”
hiloEstado2() → Estado 2 “Mostrar Secuencia”
hiloEstado3() → Estado 3 “Lectura de Pulsaciones”
hiloEstado4() → Estado 4 “Pausa Breve”
hiloEstado5() → Estado 5 “Pausa Manual”
hiloEstado6() → Estado 6 “Fase Superada”
hiloEstado7() → Estado 7 “Victoria”
```

LISTA DE MANEJADORES DE EVENTOS GPIO

int callback_inicio(int gpio)

- Filtra rebotes (descarta primeras 2 interrupciones).
- Si el botón sigue presionado y stateMgr.getState()==0:
 - buzzer.reproducirConfirmacion()
 - Reinicia secuencia y añade primer color.
 - stateMgr.changeState(1)

1. pseudocódigo

```
callback_inicio(gpio):
    static edgeCount = 0
    if edgeCount < 2:
        edgeCount += 1
        return
    if botones.leer_boton(INICIO) == false:
        return
    if stateMgr.getState() == 0:
        buzzer.reproducirConfirmacion()
        esperar 100 ms
        secuencia.reiniciar()
        secuencia.añadirColor()
        buzzer.reproducirSilencio() // detiene melodías intro
        buzzer.keepPlaying = false // señal para parar hilo interno
        stateMgr.changeState(1)
```

int callback_pausa(int gpio)

- Filtra rebotes.
- Lee estado actual $s = \text{stateMgr.getState}()$.
- Si $s == 4$: pausa manual $\rightarrow \text{stateMgr.changeState}(5)$.
- Si $s == 5$: reanuda $\rightarrow$ apaga LEDs y $\text{stateMgr.changeState}(2)$.

1. pseudocódigo

```
callback_pausa(gpio):
    static edgeCount = 0
    if edgeCount < 2:
        edgeCount += 1
        return
    if botones.leer_boton(PAUSA) == false:
        return
    s = stateMgr.getState()
    if s == 4:
        buzzer.reproducirConfirmacion()
        stateMgr.changeState(5)
    else if s == 5:
        buzzer.reproducirConfirmacion()
        leds.apagarTodos()
        esperar 100 ms
        stateMgr.changeState(2)
```

DESCRIPCIÓN DE OBJETOS Y CLASES PROPIAS DEL PROYECTO

Clase / Objeto	Ubicación	Responsabilidad principal	Métodos y atributos más relevantes
StateMonitor	src/state_monitor/stateMonitor.hpp/.cpp	Coordinar la máquina de estados global del juego y la sincronización entre hilos.	waitState(threadConf*), changeState(int), getState(). Tabla interna de manejadores de transición.

threadConf	src/state_monitor/threadConf.hpp	Estructura auxiliar para que cada hilo declare en qué estados debe despertarse.	addState(int), containsState(int), setArg(void*), getArg().
SimonBotones	src/simon_librerias/SimonBotones.hpp/.cpp	Lectura de los seis botones arcade y gestión de interrupciones.	Enum COLOR, leer_boton(COLOR), configurar_interrupcion_inicio, configurar_interrupcion_pausa. Array interno GPIOuniv* inButton[6].
SimonLeds	src/simon_librerias/SimonLeds.h/.cpp	Control de los seis LEDs (cuatro colores + START + PAUSE).	Enum LedColor, encenderLed, apagarLed, parpadearLed, encenderLedsJuego, encenderTodos, apagarTodos. Array privado de pines GPIO.
SimonBuzzer	src/simon_librerias/SimonBuzzer.hpp/.cpp	Generar tonos y melodías mediante un pin PWM.	Enum COLOR; reproducirColor(COLOR, dur_ms); reproducirMelodiasIntro() (melodía en bucle para el estado de espera); reproducirConfirmacion, reproducirFallo, reproducirAcierto, reproducirInicio, reproducirFaseExtra, reproducirVictoria. Atributo PWM pwm.
SimonSecuencia	src/simon_librerias/SimonSecuencia.hpp/.cpp	Generar, almacenar y verificar la secuencia de colores.	reiniciar(), añadirColor(), getColor(i), getNivelActual(), getNivelMax(). Vector privado std::vector<int> secuencia, entero max_niveles.
SimonDial	src/simon_librerias/simon_dial.hpp/.cpp	Leer un potenciómetro (ADC) y mover un micro-servo marcador.	actualizar(), getNivel(). Parámetros estáticos de servo (period, sval_min, sval_max). Usa un pin PWM y un canal ADC.
ADC, GPIOuniv, PWMuniv	src/ADC/, src/GPIOuniv/, src/PWMuniv/	Drivers de bajo nivel para acceso a ADC, GPIO y PWM en la BeagleBone Black.	Métodos de inicialización, lectura/escritura de registros; utilizados por las clases de más alto nivel.

Implementación

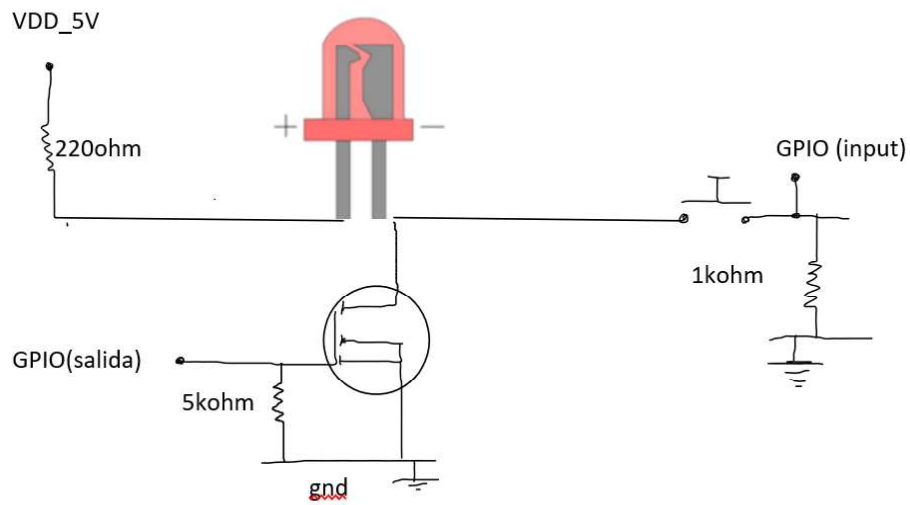
TABLA CONEXIONES BBB

Función	Elemento	Pin BBB	Modo	Observaciones
Botones GPIO	Botón Rojo	P8_7	Entrada digital	
	Botón Verde	P8_9	Entrada digital	
	Botón Amarillo	P8_11	Entrada digital	
	Botón Azul	P8_15	Entrada digital	
	Botón INICIO	P8_17	Entrada digital	
	Botón PAUSA	P8_27	Entrada digital	
LEDs GPIO	LED Rojo	P8_8	Salida digital	Con resistencia limitadora (220Ω)
	LED Verde	P8_10	Salida digital	Con resistencia limitadora (220Ω)
	LED Amarillo	P8_12	Salida digital	Con resistencia limitadora (220Ω)
	LED Azul	P8_16	Salida digital	Con resistencia limitadora (220Ω)
	LED INICIO	P8_18	Salida digital	Con resistencia limitadora (220Ω)
	LED PAUSA	P8_14	Salida digital	Con resistencia limitadora (220Ω)
PWM Buzzer	Buzzer	P9_14	Salida PWM	Con NPN
ADC (Analógicas)	potenciómetro Tiempo	P9_39 (AIN0)	Entrada analógica	Rango 0-1.8V
	potenciómetro Velocidad	P9_38 (AIN3)	Entrada analógica	Rango 0-1.8V
PWM Servos	Servo Tiempo	P9_22	Salida PWM	Alimentar servo con 5V externos
	Servo Velocidad	P8_19	Salida PWM	Alimentar servo con 5V externos

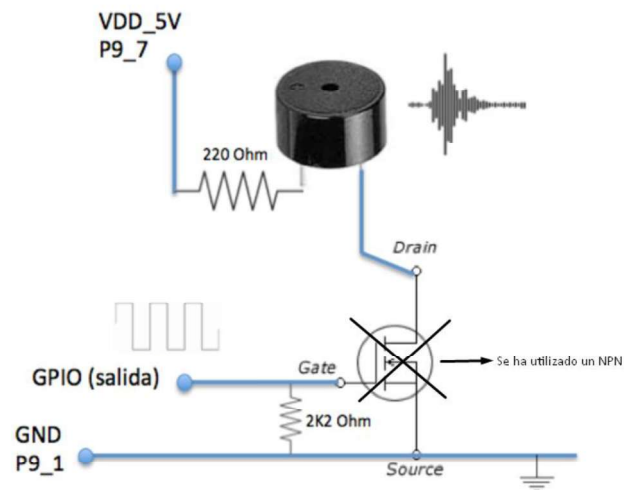
CIRCUITOS ELECTRÓNICOS

Para poder llevar a cabo las conexiones electrónicas se han utilizado los siguientes circuitos.

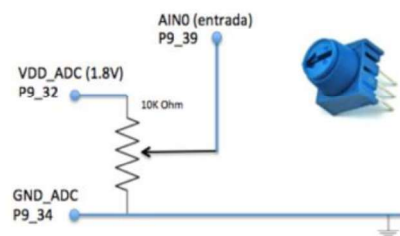
Leds + botones



Zumbador

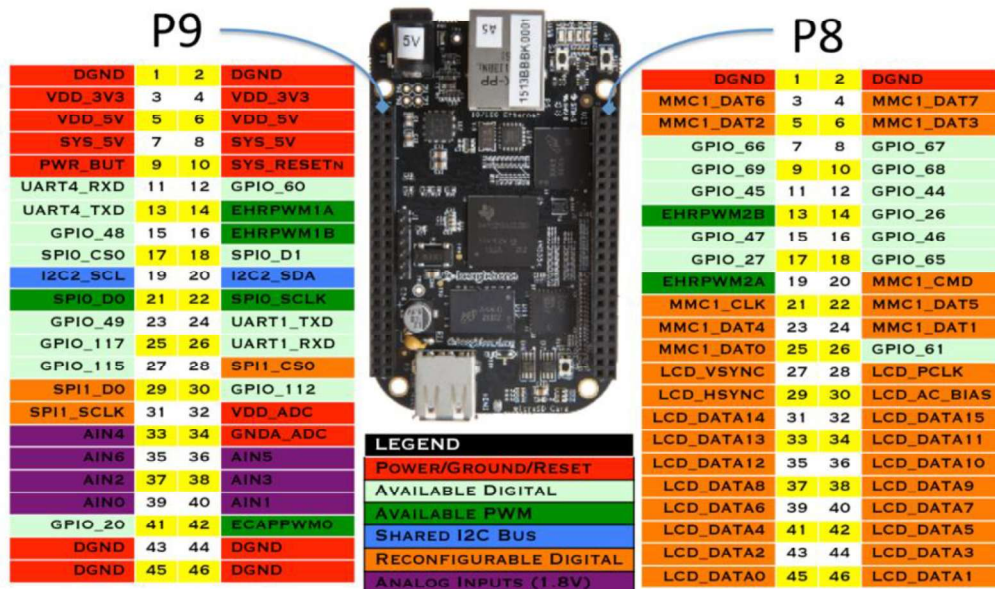


Diales





Por último, para poder seguir correctamente las conexiones de los GPIOs, PWM, señales analógicas, por decir algunas correspondientes se ha utilizado la imagen que muestra la asignación de los pines de conexión.

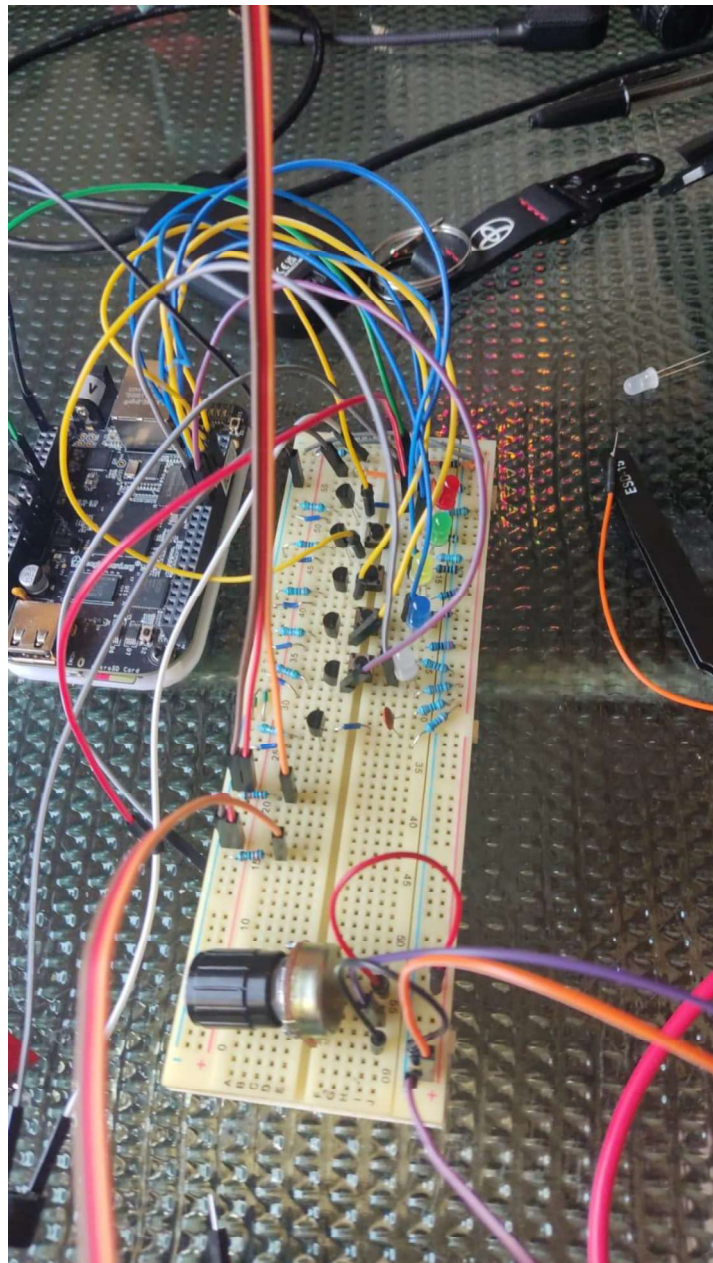
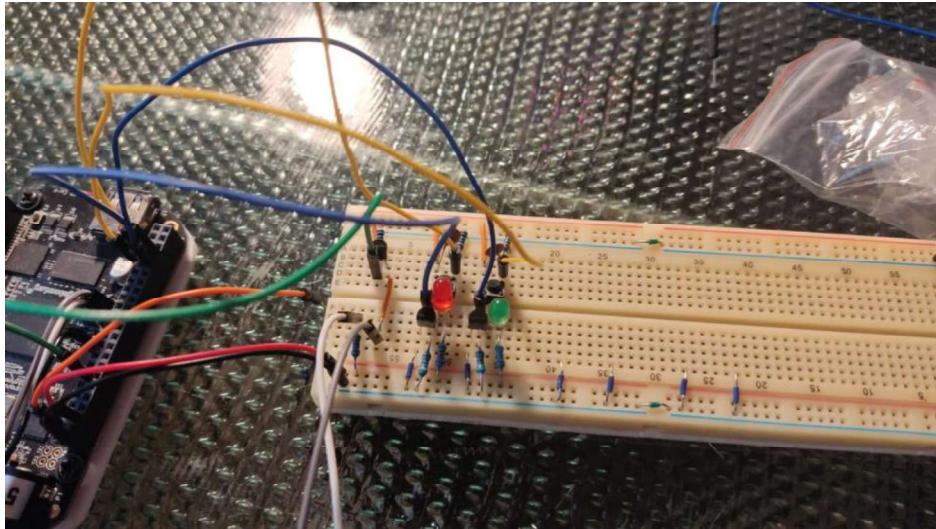


PREPARACIÓN DE LA PARTE ELECTRONICA Y LA CAJA

Preparación de la electrónica durante la fase de desarrollo

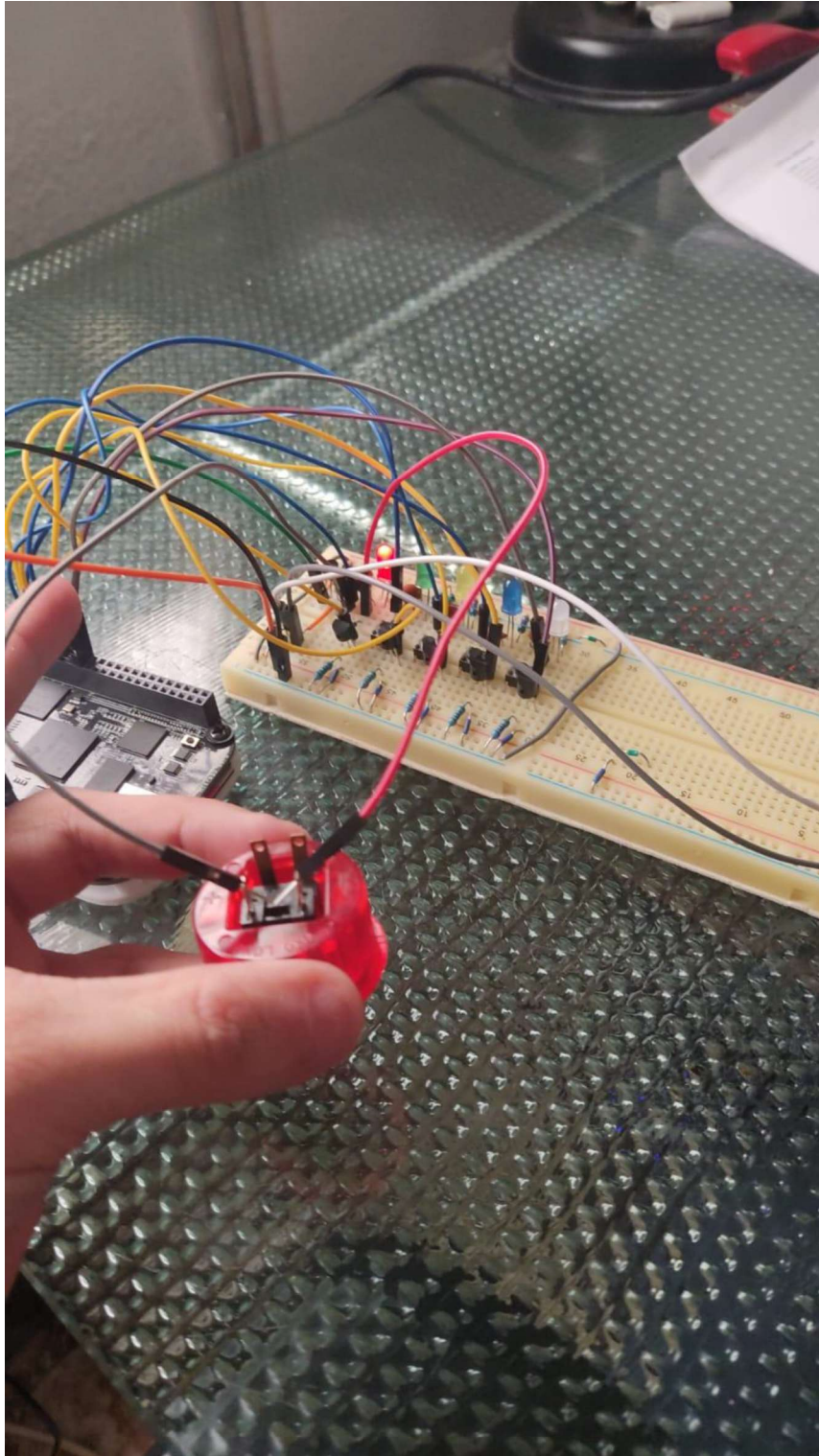
Para realizar las pruebas iniciales se ha tratado de organizar todo el circuito dentro de una única protoboard.

Se muestra un ejemplo a continuación:



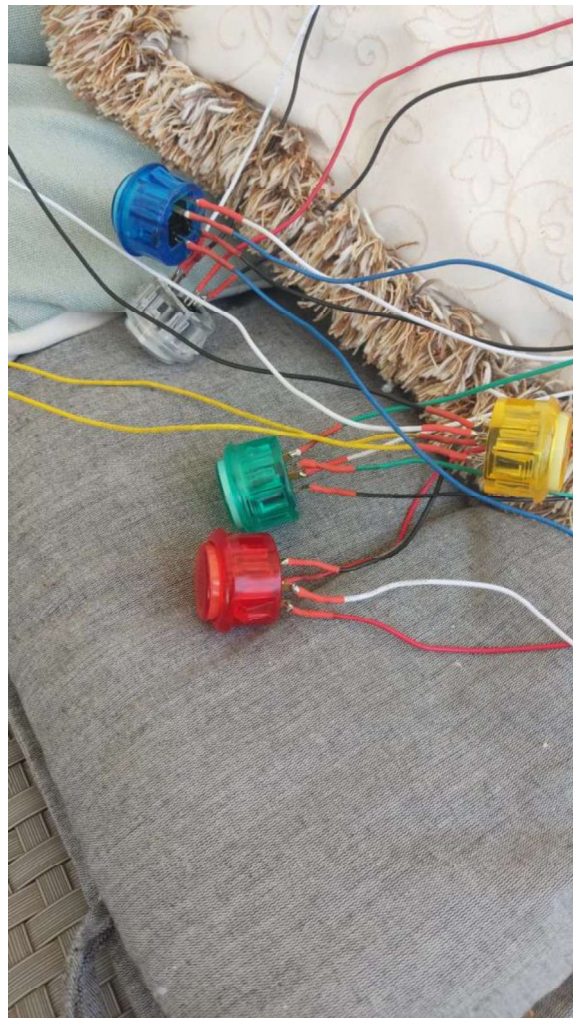
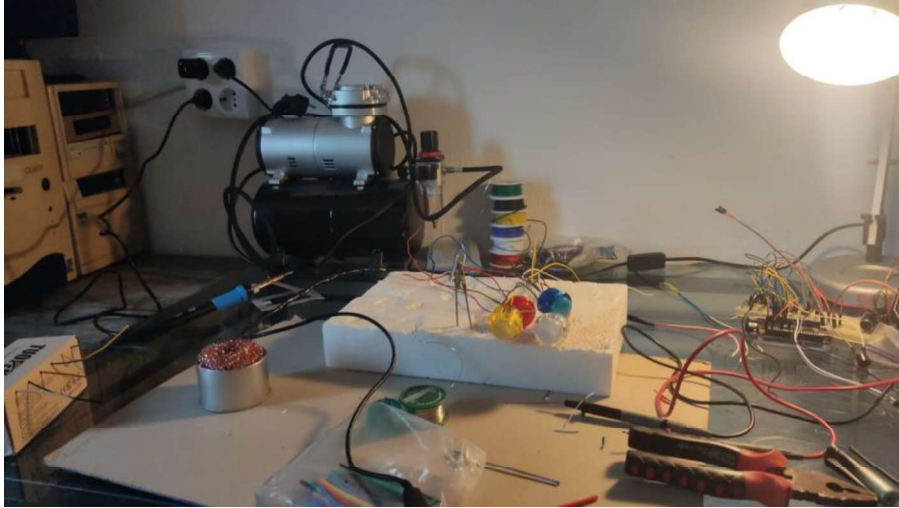
Se comprobó que las pruebas se realizaban correctamente utilizando los botones y LEDs de prueba. Sin embargo, se anticipó que, al montar la caja y conectar los botones arcade, sería recomendable distribuir el circuito en dos protoboards para facilitar la detección de posibles fallos.

Una vez recibidos los botones arcade, se realizaron pruebas con uno de ellos para verificar su correcto funcionamiento, como se muestra en la siguiente imagen.

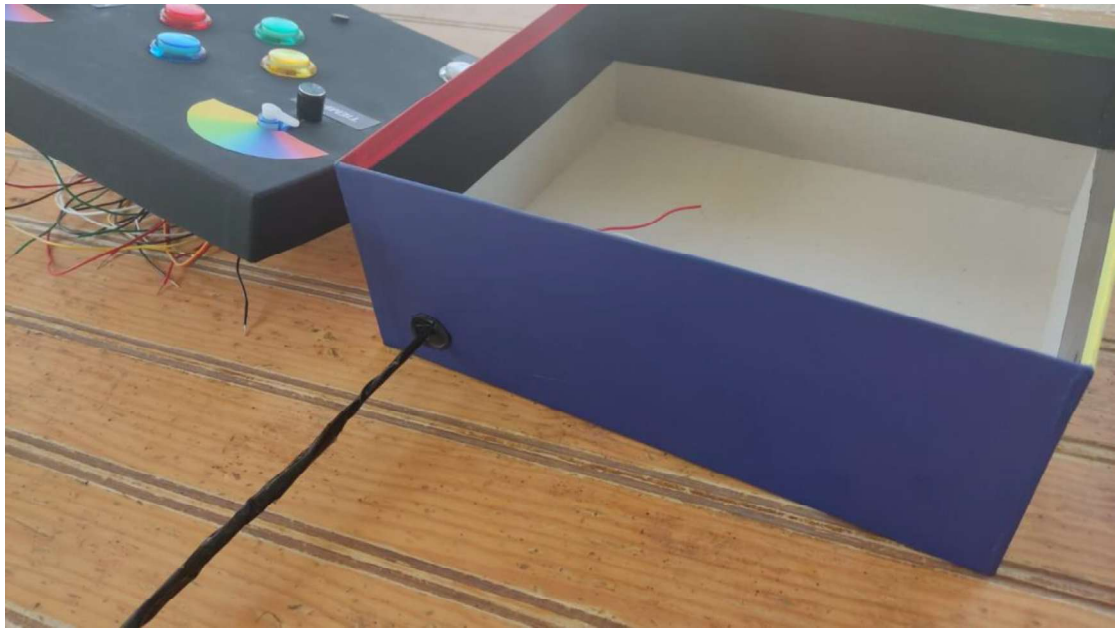


Preparación de los componentes y la caja

Primero para los componentes tocó soldar cables rígidos para los seis botones y los dos potenciómetros, a continuación, se muestran unas imágenes.

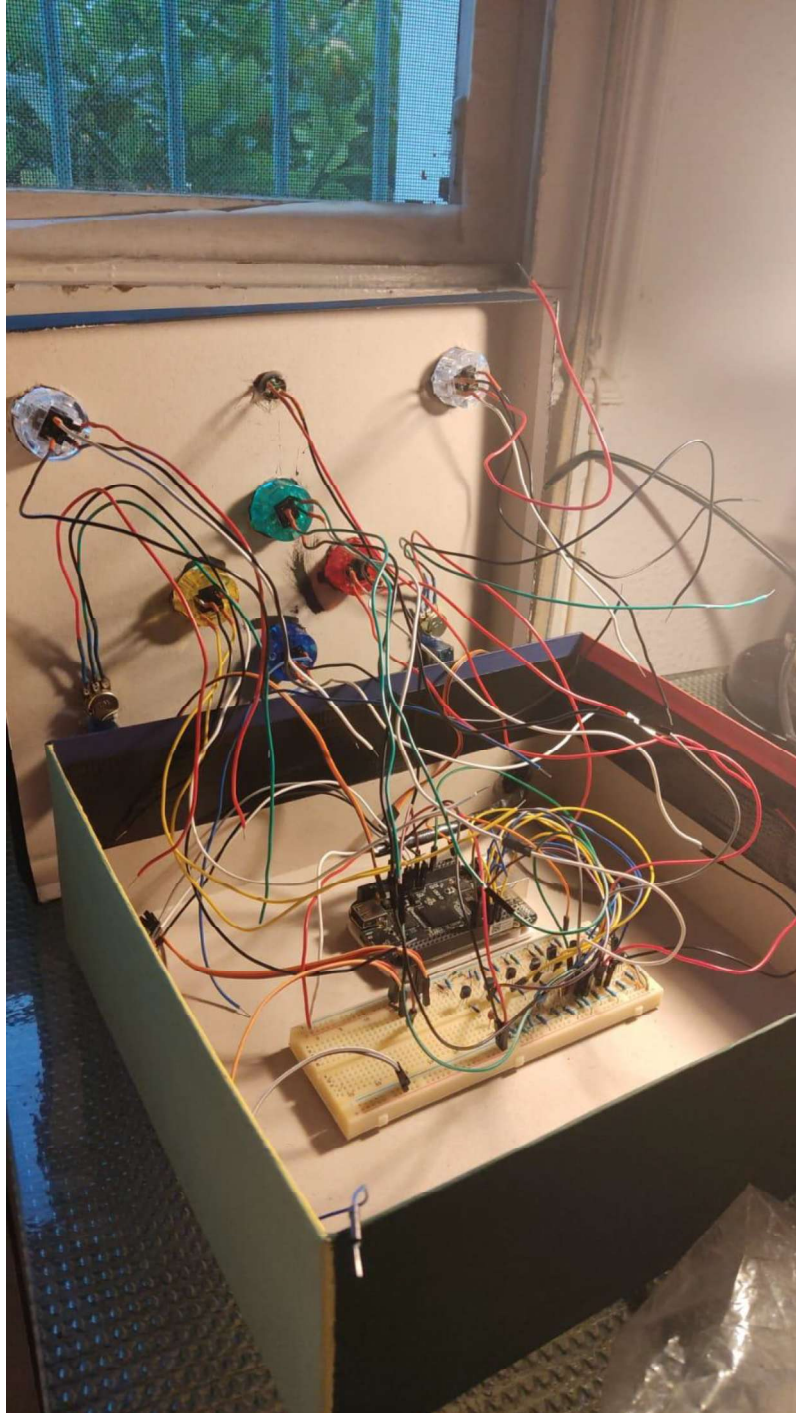


Lo siguiente que se hizo fue preparar la caja del simon, se muestra unas imágenes de ello.



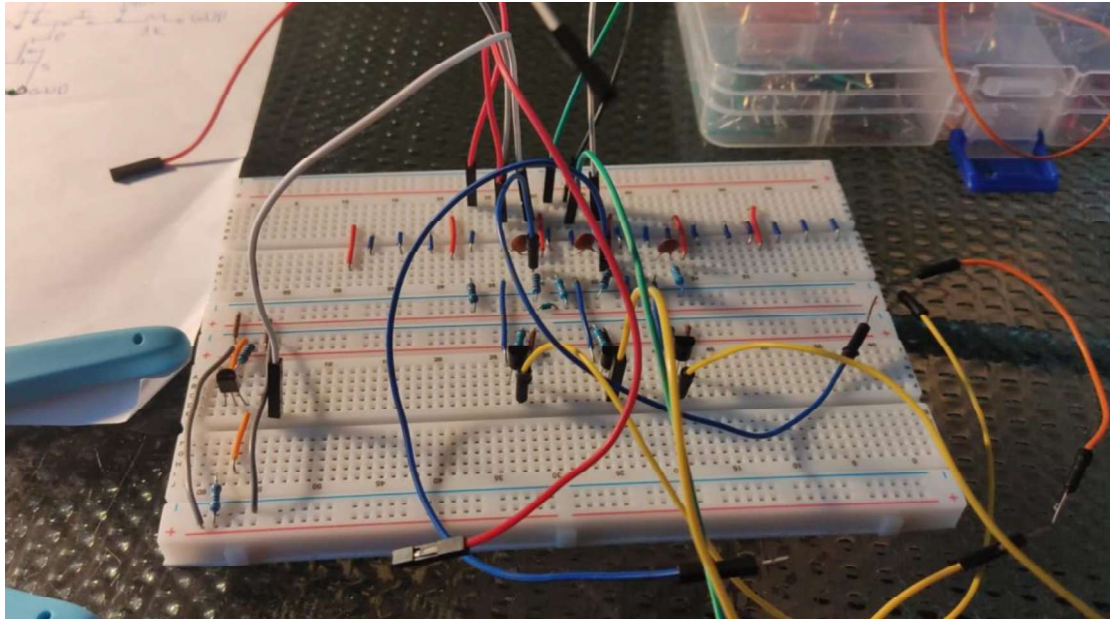
Preparación final de la caja

Primero se trató de introducir la protoboard como se mostró anteriormente, pero al momento de hacer las conexiones de todos los componentes, se encontraron problemas porque se soltaban algunos cables, y era complejo saber donde iban, como se muestra a continuación.



Por tanto, se decidió separar el circuito en dos protoboard para poder testear con mayor facilidad, y así encontrar los fallos que se encontrasen en la electrónica.

Se muestra un ejemplo de ello.



Pruebas de software

Antes de ensamblar y preparar el programa principal se fueron preparando los objetos de los distintos componentes a utilizar, y se fueron preparando distintos scripts para poder verificar el correcto comportamiento de los componentes por separado, a continuación, se muestra una tabla comentando los distintas pruebas que se realizaron.

Archivo de test	Objetivo de la prueba
main_botones_sonido.cpp	Verificar que cada botón arcade dispara inmediatamente el tono correcto en el buzzer; la ejecución termina al pulsar PAUSA.
main_botones_test.cpp	Comprobar el funcionamiento básico de los seis botones: al pulsar uno se escribe en consola su identificador; finaliza con PAUSA.
main_led_test.cpp	Ensayar la lógica de encendido y parpadeo de LEDs mediante comandos de consola: permite seleccionar un LED, alternar parpadeo con distintos tiempos y comprobar que la conducción a través de los MOSFET funciona sin parpadeos indeseados.
testDiales.cpp	Leer en tiempo real los dos potenciómetros mostrando en consola el ángulo y el nivel (0-3) a la vez que mueve el micro-servo indicador asociado a cada dial.
main_simon_nivelVel_test.cpp	Medir el efecto de los diferentes niveles de velocidad (forzados por software) sobre los retardos de la secuencia sin usar los diales.
main_simon_nivelVel_tiempo_test.cpp	Igual que el anterior, pero incluye la prueba de timeout entre pulsaciones combinando niveles de velocidad y de tiempo.
test_interrupciones_botones.cpp	Validar las interrupciones GPIO de START y PAUSE mostrando en consola los eventos y simulando un pequeño cambio de estado.
main_funcionando.cpp	Prueba de integración completa: ejecuta todo el flujo del juego con diales, buzzer, LEDs, botones e interrupciones, asegurando que los cambios de estado y la lógica de validación funcionan conjuntamente sin bloqueos ni desbordes de secuencia.

PROBLEMAS ENCONTRADOS

Problemas encontrados – Parte electrónica

Durante el desarrollo del proyecto Simon se presentaron varios problemas relacionados con el montaje electrónico. Uno de los más frecuentes fue que algunos LEDs no encendían correctamente. En muchos casos, esto se debía a que el transistor asociado no funcionaba adecuadamente, por lo que hubo que revisar y reemplazar componentes.

Tras realizar el montaje final en la caja, uno de los LEDs dejó de funcionar completamente. Después de varias comprobaciones, se detectó que el problema estaba en una soldadura suelta. Fue necesario resoldar la conexión, pero hasta que se localizó el fallo se invirtió bastante tiempo en diagnóstico.

También se observó que ciertos LEDs se encendían con menor intensidad de lo esperado. Finalmente, se descubrió que una de las conexiones de masa (GND) no estaba bien conectada, lo que causaba un mal funcionamiento intermitente.

Por último, otro problema aún no resuelto del todo es el comportamiento errático de uno de los servomotores. En ocasiones comienza a moverse sin control, y todo indica que se debe a un falso contacto en el cableado, ya que este comportamiento se activa al mover la tapa de la caja. Se sospecha de una mala conexión física en algún punto del circuito.

Problemas encontrados – Parte software

Durante el desarrollo del programa del Simon, el primer gran reto fue diseñar correctamente los objetos y estructurarlos adecuadamente en C++. Esta etapa fue la que más tiempo consumió en todo el proyecto, ya que implicó definir bien las responsabilidades de cada clase y su implementación modular.

Una vez se empezó a crear los objetos, los primeros problemas surgieron con el buzzer: al principio no funcionaba correctamente porque se estaba utilizando un modelo de zumbador inadecuado. Tras sustituirlo por uno compatible con PWM, el problema se resolvió.

Posteriormente, los diales también dieron bastantes complicaciones. Al principio el movimiento del servo era brusco y los valores se leían de forma errática. Fue necesario ajustar varios parámetros (como el periodo y los valores mínimo y máximo del PWM) hasta que los servos SG90 funcionaron de manera fluida y precisa.

Con todos los objetos ya definidos, se realizó una primera prueba del juego Simon sin estados ni hilos, simplemente ejecutando el simon sin inicio, solo haciendo un de los botones del Simon y el zumbador. Al comprobar que todo funcionaba correctamente, se pasó a organizar el programa en estados e hilos.

En esta nueva fase, la implementación de los primeros estados (inicio, secuencia y lectura de botones con sonidos) se desarrolló sin incidencias. Sin embargo, el principal problema surgió al intentar implementar el estado de **pausa**. Al pulsar el botón PAUSA, el juego cambiaba correctamente al estado correspondiente, pero al volver a pulsarlo no regresaba al juego y quedaba bloqueado. En ese momento, todos los botones se gestionaban mediante **polling**, lo cual no era eficaz para este caso.

La solución fue migrar los botones de inicio y pausa a gestión por **interrupciones GPIO**. Este cambio también presentó dificultades: al principio las interrupciones se activaban solas debido a ruido eléctrico inicial, por lo que se tuvo que implementar un pequeño sistema de filtrado que descartara los primeros pulsos tras el arranque. Resolver este comportamiento llevó casi un día completo. Posteriormente, se detectó que las pulsaciones por interrupción a veces respondían con lentitud, lo cual requirió revisar cuidadosamente los retardos y el control de flags.

Una vez estabilizadas las interrupciones, se volvió a revisar el comportamiento de los diales. Uno de ellos seguía presentando fallos y movía su servo de forma errática, como se detalló en los problemas de hardware, y se sospecha de un fallo físico en el cableado.

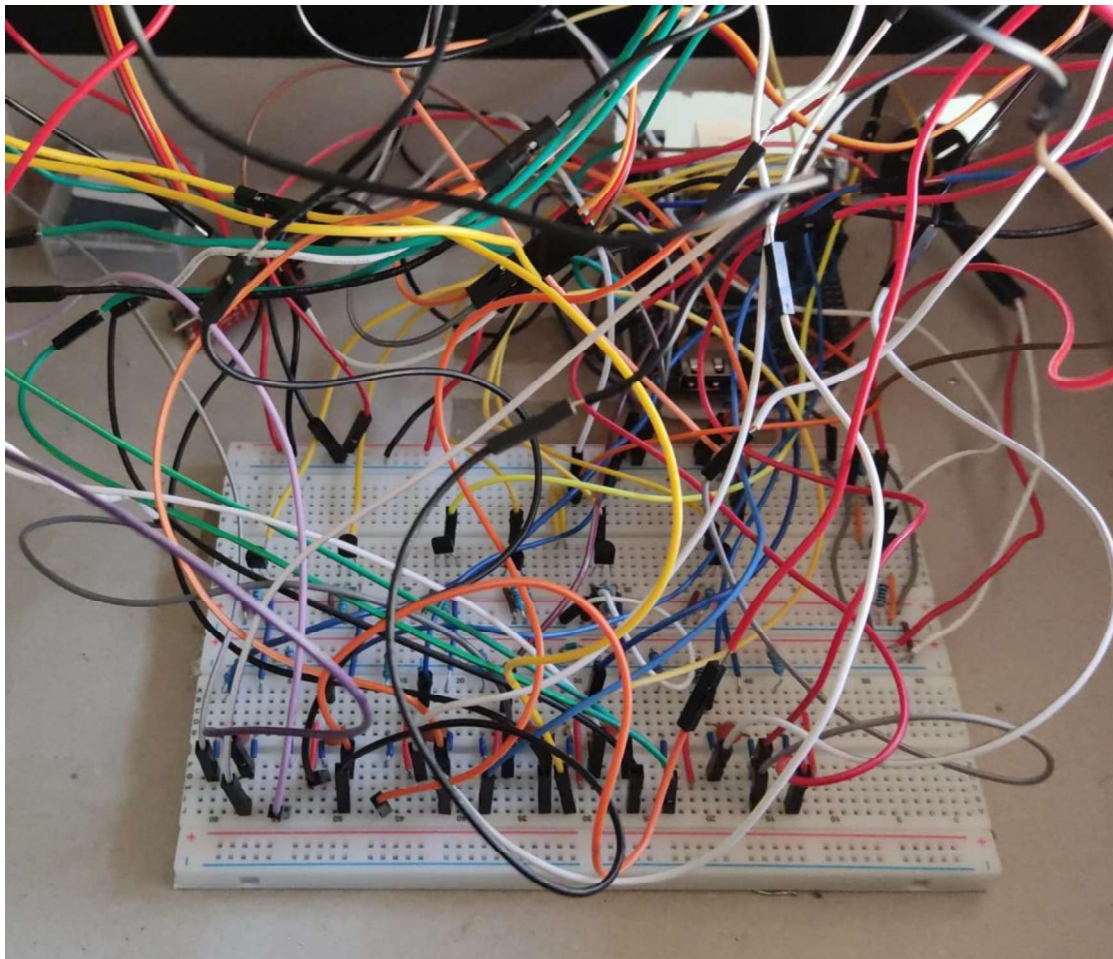
Tras eso, se perfeccionaron los sonidos de colores y la lógica de la secuencia, buscando una experiencia más pulida y coherente. También se implementó la melodía de fondo que suena en el estado de inicio. Esta parte funcionaba bien, hasta que se introdujeron melodías más largas: cuando el jugador comenzaba la partida, la melodía aún estaba sonando, lo que provocaba bloqueos o retardos.

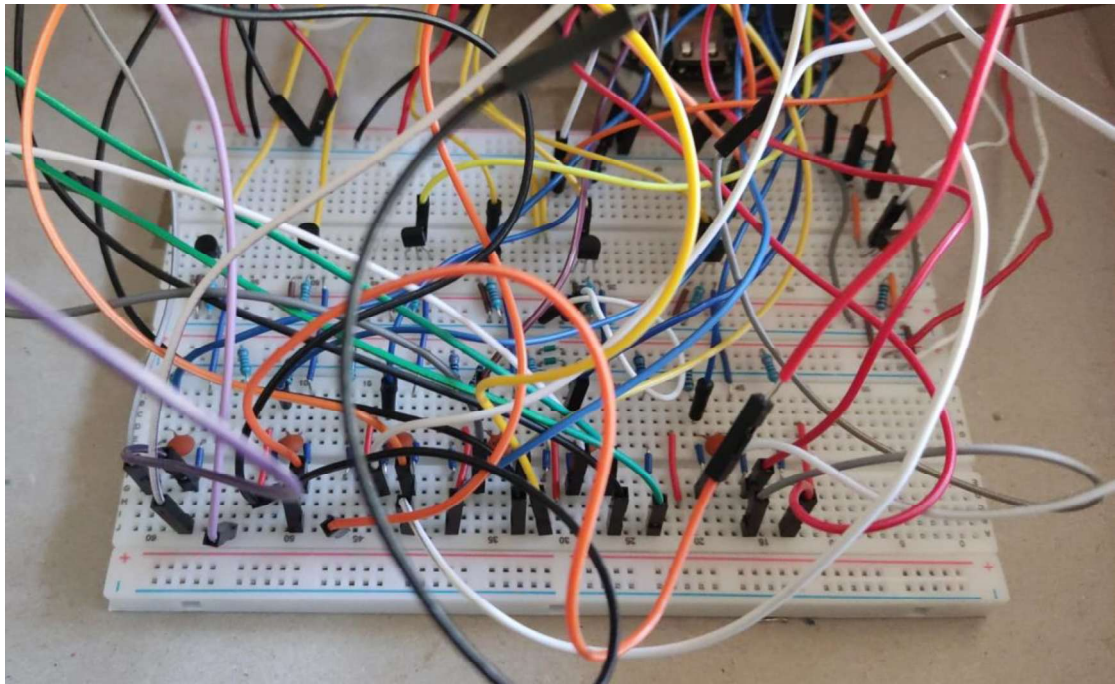
La solución fue modificar la función de reproducción de melodías para que comprobara una variable booleana (`keepPlaying`). Si esta se ponía a `false`, el bucle de la melodía se detenía inmediatamente, permitiendo al juego cambiar de estado sin esperar a que la melodía finalizara por completo.

Por último, otro reto fue lograr que el programa se ejecutara automáticamente al encender la BeagleBone Black. Al principio no estaba claro cómo hacerlo, pero finalmente se resolvió fácilmente mediante el uso de un servicio `systemd`.

RESULTADO FINAL

Por ultimo se muestra como ha quedado el Simon. Primero se muestra como ha quedado por dentro.





Por último, se muestra el juego Simon, que es lo que va a ver el usuario.







Junto con la documentación del código, se han incluido dos vídeos que muestran el funcionamiento del juego Simon: el primero presenta todas sus funcionalidades, y el segundo muestra la fase final tras alcanzar la victoria.